

세미나 1

발표자: 정해찬

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- Kotlin 코드 이해하기
- 안드로이드 앱의 구성 요소
- 과제 안내

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- Kotlin 코드 이해하기
- 안드로이드 앱의 구성 요소
- 과제 안내

오리엔테이션

- 교육팀 소개
- 세미나 소개
- 여러분의 자기소개

교육팀 소개



정해찬

- 와플 22.5기 & 컴공 24학번
- SNUTT Android(2025.03.~)
- 23.5기 Android 세미나장



권성안

- 와플 23.5기 & 산공 24학번
- 행사 Android(2026.03.~)
- 2026년 와플 운영팀

세미나 소개

- 월요일 오후 6시 ~ 7시 반
- 9/14, 9/28, 10/12, 11/2, 11/16, 11/30
- 세미나 수료 조건
 - 세미나 6회 출석
 - 불가피한 사정이 있을 경우 미리 말씀해 주시면 동영상으로 대체 가능합니다.
 - 시험기간(10/26)에는 동영상 녹화로 대체 예정입니다.
 - 과제 6회 제출 및 통과
 - 과제 기한은 다음 세미나 전까지이며, grace day는 없습니다.
 - 불가피한 사정으로 기한을 초과할 것 같은 경우, 3일 전까지 말씀해 주세요.

여러분의 소개

- 친하게 지내요!
- 학과와 이름
- 관심사
- 와플에 들어온 이유
 - 안드로이드를 수강하는 이유?
- 개발 경험 (중요)
 - 할 줄 아는 언어?

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- Kotlin 코드 이해하기
- 안드로이드 앱의 구성 요소
- 과제 안내

AI 시대의 개발

- 개발(SWE)이 무엇인가요?
- AI로 개발하기

개발, SWE가 무엇인가요?

Software Engineering

Application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

— ISO/IEC TR 19759:2016, Software Engineering, Guide to the Software Engineering Body of Knowledge

- 말 그대로의 정의: SW를 만들고, 관리하는 것
- SW를 설계 => 제작 => 테스트 => 관리하는 사이클의 모든 부분
- SWE의 특징: 제작하다가 설계를 바꿀 수 있다.
 - 장점: 설계를 언제든지 바꿀 수 있다.
 - 단점: 설계를 계속 바꾸게 된다.

개발, SWE가 무엇인가요?

- SW를 설계 => 제작 => 테스트 => 관리하는 사이클의 모든 부분
- 제작 기술과 인적 관리, 프로세스가 모두 SWE 중요한 요소이다!
- 이 세미나에서는 앱 제작 기술에 대해서만 이야기합니다.
- 이후 팀 활동에서 인적 관리 및 프로세스의 중요성을 배우시게 될 겁니다.

코드의 조건

- 작성하시는 코드는 다음 조건들을 만족해야 합니다.

1. 요구사항을 정확히 만족할 것

- 엣지 케이스가 없을 것

2. 다른 사람(AI)가 이해하기 쉬울 것

- 책임 분리가 명확하고 이름이 명시적이며, 깔끔할 것

- 나중에 이야기할 기회가 많을 거예요

3. 확장하거나 변경하기 쉬울 것

4. 일관적이고 테스트가 쉬울 것

5. 성능이 좋을 것

AI 시대의 개발

- AI가 발전하면서 바뀐 점
 - 와플 지원자 수가 1/3이 됐다.
 - 열심히 앱 만들어서 에타에 올리면 " 그거 AI 딸각하면 되는 거 아님?"이 달린다.
 - 개발자 신입 채용이 안 열린다.
- 코드 작성에 드는 자원이 극도로 작아짐
 - 이제 병목은 인간의 코드 검수 및 상호 리뷰
 - 이것도 AI 시키는 workflow도 있는데 어차피 최종 검토 후 책임은 본인이 지는 거니까요...

AI 이용해서 개발하기

- 코드 작성: AI
- 코드 아키텍처를 어떻게 구성할지 고민하고 결정하기, 혼자서 결정이 어렵다면 팀원들과 토의하기, 각 커밋이 어떤 내용을 담게 할지 판단하기, AI에게 명료하게 작성할 코드를 지시하기, AI가 쓴 코드를 한 줄 한 줄 읽어보고, 충분히 좋은 코드인지 판단하고, 만약 아니라면 다시 수정하기 등: 인간
- **AI 코드 한 줄도 빼놓지 말고 전부 다 읽으세요**

AI 이용해서 개발하기

- CLI 사용하기
 - 학교에서 주는 Edu Plan에 Codex가 포함되어 있습니다!
 - 아마 세미나에는 충분할 거예요
 - <https://chatgpt.com/codex/>
- AI가 변경한 코드는 어떻게 검토하나요?
 - IDE에서 확인하기
 - Git 프로그램에서 확인하기
 - Git 기억나시나요?
 - 변경을 잘 처리하기 위해 만들어진 기술이기 때문에, AI가 변경한 코드 보기에도 편합니다.

AI 이용해서 개발하기

- 스킬 사용하기
 - 반복되고 단순한 작업을 스킬로 설정하면, 매번 귀찮게 시킬 필요가 없습니다!
 - 객체 테스트 작성, UI 테스트 작성 등
 - 다른 동아리 ppt 제작, 파일 업로드 등에도 쓰는 중입니다
- AGENTS.md
 - 에이전트에게 프로젝트 전체에 대해서 지켜야 할 작업 규칙을 알려주는 파일
 - 간단하게는 커밋 메시지 형식, “내가 말하기 전까지는 커밋하지 마”부터
 - 크게는 앱의 아키텍처, 각 부분의 역할 분리 등을 안내할 수 있습니다.

AI 이용해서 개발하기

- 개발자의 마인드셋
 - 더 편하게 / 자동화 할 수 있는 부분이 있는가?
 - AI에게 더 위임할 수 있는 부분이 있는가?
 - 더 빠르게 많은 작업을 할 방법이 없는가?
- 만약 있다면 그렇게 해도 좋습니다!
- 최대한 빠르게, 각 커밋이 좋은 코드를 담고 있기만 하면 돼요!

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- Kotlin 코드 이해하기
- 안드로이드 앱의 구성 요소
- 과제 안내

Git에 대해서

- 많은 사람이 하나의 파일을 변경해야 합니다
- zip 파일을 주고받으면, 수많은 변경을 하나로 모으는 것이 불가능합니다.
- 그래서 등장한 것이 Git

로컬 저장소

- 우리는 각자의 컴퓨터에 '로컬 저장소'를 가지고 있습니다.
- Github 서버에는 우리가 공동으로 작업하는 '원격 저장소'가 있습니다.
- 우리가 코드를 수정하면, 그 변경이 자동으로 기록되지는 않습니다.
- "어떤 파일들에 발생한 변경을 기록하겠다"는 명령이 add입니다.
- "발생한 변경들을 로컬 저장소에 기록해라"라는 명령이 commit입니다.
- 커밋이 우리가 주고받는 "변경"의 단위입니다.

로컬 저장소

- "어떤 파일들에 발생한 변경을 기록하겠다"는 명령이 add입니다.
- "발생한 변경들을 로컬 저장소에 기록해라"라는 명령이 commit입니다.



test1.py



test2.py



test3.py

```
git init
git add test1.py
git add test2.py
git commit -m "first commit"
echo "1234" >> test2.py
echo "1234" >> test3.py
git commit -m "second commit "
```

추적되면 안 되는 파일들은 .gitignore에!

원격 저장소

- 내 로컬 저장소의 변경(커밋)을 원격 저장소로 보낼 수도,
 - 원격 저장소의 변경(커밋)을 로컬 저장소로 받아올 수도 있습니다.
-
- push: 로컬 -> 원격
 - fetch: 원격 -> 로컬, 로컬에 origin/main같은 브랜치가 생김
 - pull: 원격 -> 로컬, 원격 저장소의 변경이 로컬의 main으로 합쳐짐

원격 저장소

- 내 로컬 저장소의 변경(커밋)을 원격 저장소로 보낼 수도,
- 원격 저장소의 변경(커밋)을 로컬 저장소로 받아올 수도 있습니다.
- push: 로컬 -> 원격
- fetch: 원격 -> 로컬, 로컬에 origin/main같은 브랜치로 받아옴
 - 로컬에 있던 main 브랜치와는 별도
- pull: 원격 -> 로컬, 원격 저장소의 변경이 로컬의 main으로 합쳐짐
- clone: 원격 저장소를 복사해 로컬 저장소 생성

브랜치

- 평행세계의 코드를 의미합니다.
- merge: A 브랜치의 변경을 B 브랜치로 합침
- merge conflict: 두 개 이상의 커밋이 같은 line을 수정하고 있을 때
 - 사람(AI)이 직접 수정해주어야 합니다.
- develop을 메인 브랜치로 설정
 - feat/server-english 브랜치를 만들어 기능을 추가 후
 - feat/server-english -> develop으로 Pull Request를 보냄
 - 팀원이 리뷰 후 develop으로 merge할 수 있음

포크

- 내 깃헙 계정에 원격 저장소를 복사해서 새로운 원격 저장소를 생성
- 작업 후 기존 원격 저장소로 PR을 보내는 것도 가능

Questions?

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- **Kotlin 코드 이해하기**
- 안드로이드 앱의 구성 요소
- 과제 안내

Kotlin 코드 이해하기

- 변수, 함수, 조건문, 반복문, 컬렉션 함수
- 객체지향 프로그래밍
 - 객체와 클래스
 - 다양한 종류의 객체들
 - 추상화, 인터페이스

Kotlin 코드 이해하기

- 변수, 함수, 조건문, 반복문, 컬렉션 함수
- 객체지향 프로그래밍
 - 객체와 클래스
 - 다양한 종류의 객체들
 - 추상화, 인터페이스

변수, 함수

- 변수

- 값을 저장해 두는 이름 붙은 메모리 공간
- Kotlin에서는 val과 var으로 선언 가능
- : 뒤에 변수의 자료형을 적어줌
 - 생략 가능

```
val seminar: String = "Android"  
var students: Int = 10
```

```
seminar = "iOS"  
students += 1
```

- 자료형

- 변수가 가질 수 있는 값의 종류
 - Int, Long, Double, Boolean, Char, String, ...
 - List, MutableList, Set, Map, Pair

변수 - 자료형

- 자료형

- List vs MutableList

- 읽기 전용 vs 수정 가능 리스트

```
val projects: List<String> = listOf("SNUTT", "식샤")
```

```
projects.add("행사")  
projects[0] = "행사"
```

```
val projects: MutableList<String> = listOf("SNUTT", "식샤")
```

```
projects.add("행사")  
projects[0] = "행사"
```

```
val numbers = mutableListOf(1, 2)
```

```
numbers.add(3)  
numbers = mutableListOf(4, 5)
```

변수 - 자료형

- null
 - 값이 없음을 나타내는 값
 - ?를 붙여 nullable 값으로 만들 수 있음

```
var friend: String? = "철수"  
friend = "민수"  
friend = null
```

```
val length: Int? = friend?.length  
val length: Int = friend?.length ?: 0  
val length: Int = friend!!.length
```

```
if (length != null){  
    length = friend.length  
}
```

변수 - 자료형

- Map
 - Python의 Dictionary: key - value 쌍을 저장
- Set
 - 집합; 중복 제거하고 싶을 때 set으로 바꿨다가 list로 바꾸기
- enum class
 - 정해진 값들 중 하나를 표현하고 싶을 때 사용
- Pair, Triple
 - 값 두/세 개를 하나로 묶을 때

```
enum class Color {  
    RED, BLUE, GREEN  
}
```

```
val color = Color.RED
```

함수

- 특정 작업을 수행하는 코드 묶음
 - 매개변수를 0개 이상 받아, 1개 이하의 값을 반환
 - 매개변수는 타입을 반드시 적어야 함
-
- 반환하는 값이 없다면 반환 타입은 Unit(없다는 뜻)
 - 기본값을 지정할 수 있음

```
fun add(a: Int, b: Int): Int {  
    return a + b  
}
```

```
val result = add(3, 5) // 8
```

함수

- 함수를 값으로 저장할 수 있음

```
val onSaveClick: () -> Unit = {  
    println("저장되었습니다.")  
}
```

```
val onDeleteClick: () -> Unit = {  
    println("삭제되었습니다.")  
}
```

```
Button(onClick = onSaveClick)  
Button(onClick = onDeleteClick)
```

```
val add: (Int, Int) -> Int = { a, b -> a + b }
```

```
println(add(2, 3)) // 5
```

함수 내용을 한 줄로 쓰고 싶다면 -> 사용

버튼이라는 객체, 혹은 함수에 이렇게 전달

조건문

- if & else
 - 조건이 참이면 if, 거짓이면 else
 - Kotlin에서는 값으로 쓸 수 있음

```
val score = 85
```

```
if (score >= 90) {  
    println("A")  
} else if (score >= 80) {  
    println("B")  
} else if (score >= 70) {  
    println("C")  
} else {  
    println("F")  
}
```

```
val age = 20
```

```
val result = if (age >= 18) {  
    "성인"  
} else {  
    "미성년자"  
}
```

```
println(result) // 성인
```

조건문

- when
 - 여러 조건을 비교할 때
 - Switch-Case?
 - 역시 값으로 쓸 수 있음

```
val grade = "B"
```

```
when (grade) {  
    "A" -> println("최우수")  
    "B" -> println("우수")  
    "C" -> println("보통")  
    else -> println("기타")  
}
```

```
val message = when (grade) {  
    "A" -> "최우수"  
    "B" -> "우수"  
    "C" -> "보통"  
    else -> "기타"  
}
```

반복문

- 같은 코드를 여러 번 실행할 때 사용
- for
 - 저는 Kotlin으로 for문을 써본 적이 한 번도 없습니다
- while
 - while도 안 써봤을걸요...?
- break: 반복문을 즉시 종료하고 탈출
- continue: 반복문을 즉시 종료하고 다음 반복으로

```
for (i in 1..5) {  
    println(i)  
}
```

1..5라고 쓰면
1부터 5까지의 IntRange를 반환

```
var count = 1
```

```
while (count <= 5) {  
    println(count)  
    count++  
}
```

컬렉션 함수

- forEach
- filter
- map
- find
- any, all, none

```
val prices = listOf(1000, 2000, 3000)
```

```
val discountedPrices = prices.map { price ->  
    price * 90 / 100  
}
```

```
println(discountedPrices) // [900, 1800, 2700]
```

```
val prices = listOf(1000, 2500, 3000, 500)
```

```
val result = prices  
    .filter { it >= 1000 }  
    .map { it * 90 / 100 }  
    .sum()
```

```
println(result)
```

- 모두 원소를 순회하며, 각각(혹은 전체)에 함수를 적용함

Kotlin 코드 이해하기

- 변수, 함수, 조건문, 반복문, 컬렉션 함수
- 객체지향 프로그래밍
 - 객체와 클래스
 - 다양한 종류의 객체들
 - 추상화, 인터페이스
- Kotlin Flow

객체지향 프로그래밍

- 객체지향 프로그래밍
 - 프로그램을 '객체'들의 상호작용으로 바라보기
 - 각 객체는 '데이터'와 '기능'을 가짐
- 필요성
 - 관련 데이터와 기능을 모아서 함께 관리, 관련 없는 데이터와 기능은 분리
 - 현실과 비슷함 (사용자, 상품, 주문 객체)
 - 같은 형태의 객체를 찍어낼 수 있음

클래스와 객체

- 클래스

- 객체를 만들기 위한 설계도

- User라는 클래스로 만들어진 객체들은
 - 이름과 나이 데이터를 주고 태어나며,
 - 자기소개를 할 수 있어야 한다.

- 객체

- 실제로 만들어진 대상
 - user1은 철수, 20살
 - user2는 영희, 21살

```
class User {  
    var name: String = ""  
    var age: Int = 0
```

객체/클래스가 들고 있는
변수는 "프로퍼티",
함수는 "메서드"

```
    fun introduce() {  
        println("저는 $name이고, $age살입니다.")  
    }  
}
```

```
val user1 = User()  
val user2 = User()
```

```
user1.name = "철수"  
user1.age = 20
```

```
user2.name = "영희"  
user2.age = 21
```

```
user1.introduce() // 저는 철수이고, 20살입니다.  
user2.introduce() // 저는 영희이고, 21살입니다.
```

클래스와 객체

- 데이터를 저장하기 위해서는 data class
 - equals, copy, toString 등 지원
- 접근 제어
 - private을 붙이면 외부에서 접근 불가
- getter, setter
 - 이 프로퍼티를 읽으려고 하면 이렇게 반환해라

```
class Person(  
    val firstName: String,  
    val lastName: String  
) {  
    val fullName: String  
        get() = "$firstName $lastName"  
}
```

```
data class User(  
    val name: String,  
    val age: Int  
)
```

```
class BankAccount(  
    val owner: String,  
    private var balance: Int  
) {  
    fun deposit(amount: Int) {  
        if (amount > 0) {  
            balance += amount  
        }  
    }  
  
    fun getBalance(): Int {  
        return balance  
    }  
}
```

클래스와 객체

- 생성자
 - 필요한 값을 생성 시 바로 요구
 - 생성 시 init 블록에 넣은 코드가 실행됨

```
class User(  
    val name: String,  
    val age: Int  
) {  
    fun introduce() {  
        println("저는 $name이고, $age살입니다.")  
    }  
}
```

```
val user = User("민수", 20)  
user.introduce() // 저는 민수이고, 20살입니다.
```

```
class User(  
    val name: String,  
    var age: Int  
) {  
    init {  
        require(age >= 0) { "나이는 음수일 수 없습니다." }  
        println("$name 객체가 생성되었습니다.")  
    }  
}
```

상속

- 기존 클래스의 메서드, 프로퍼티를 다른 클래스가 물려받는 것
- Kotlin에서는 기본적으로 final, 상속받기 위해서는 open을 붙여야 함
- 메서드 일부를 재정의할 수 있음
 - override

```
open class Animal(  
    val name: String  
) {  
    fun introduce() {  
        println("이름은 $name입니다.")  
    }  
  
    open fun sound() {  
        println("소리를 냅니다.")  
    }  
}
```

```
class Dog(name: String) : Animal(name) {  
    override fun sound() {  
        println("멍멍")  
    }  
}  
  
class Cat(name: String) : Animal(name) {  
    override fun sound() {  
        println("야옹")  
    }  
}
```

인터페이스

- “이 인터페이스를 구현하는 클래스는 이런 함수를 가져야 한다”
 - 그 함수가 어떻게 구현되는지는 신경 X

```
interface Flyable {  
    fun fly()  
}
```

```
class Bird : Flyable {  
    override fun fly() {  
        println("날개를 퍼덕이며 날아갑니다.")  
    }  
}
```

```
class Airplane : Flyable {  
    override fun fly() {  
        println("엔진으로 날아갑니다.")  
    }  
}
```

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- Kotlin 코드 이해하기
- 안드로이드 앱의 구성 요소
- 과제 안내

안드로이드 앱의 구성 요소

- 안드로이드 프레임워크
- Android Studio 프로젝트 구조
- 안드로이드 앱의 구성 요소 4가지
- Activity Lifecycle
- Context

안드로이드 앱의 구성 요소

- 안드로이드 프레임워크
- Android Studio 프로젝트 구조
- 안드로이드 앱의 구성 요소 4가지
- Activity Lifecycle
- Context

안드로이드 프레임워크

- main.py를 실행하는 것 vs 안드로이드 앱을 실행하는 것
- 이미 훌륭한 사람들이 만들어 둔 프레임워크를 사용
- 우리는 각 화면, 각 데이터에 해당하는 코드를 작성해서 채워 둡니다
 - 안드로이드 프레임워크가 적절한 곳에 그 코드들을 꽂아, full-app을 완성합니다
- 빈칸이 뚫려 있는 학습지를 받아, 알맞게 채워 넣기만 하면 됩니다

안드로이드 프레임워크

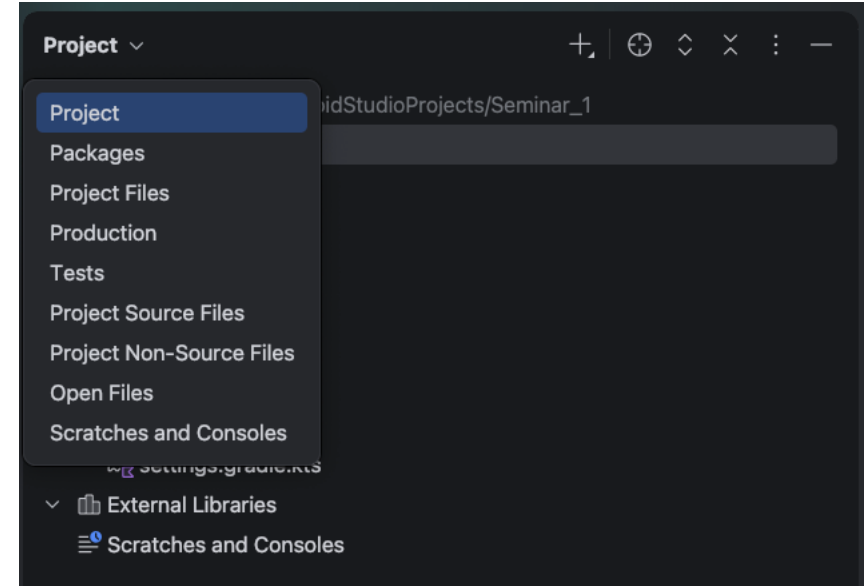
- 앱을 켜면 우리 코드가 실행되는 게 아니라,
 - Android OS가 우리가 채워 둔 '첫 화면'을 찾아서 띄워줍니다
 - 이외에도 이벤트, 서비스 등이 발생하면
 - Android OS가 해당하는 우리 코드를 찾아서 실행해 줍니다.

안드로이드 앱의 구성 요소

- 안드로이드 프레임워크
- Android Studio 프로젝트 구조
- 안드로이드 앱의 구성 요소 4가지
- Activity Lifecycle
- Context

Android Studio 프로젝트 구조

- Project가 실제 파일 구조
- Gradle
 - 빌드 도구
 - 우리가 쓴 코드를 잘 끼워맞춰서 앱으로 쌓아줍니다
 - libs.versions.toml
 - 사용하는 라이브러리들의 버전을 모아놓는 파일
 - build.gradle.kts
 - app 수준과 프로젝트 수준이 있습니다
 - gradlew
 - 여러 가지 빌드 도구들을 제공합니다



Android Studio 프로젝트 구조

- res 폴더
 - 앱에서 사용하는 아이콘, 그림 등 리소스 파일
- AndroidManifest.xml
 - OS에 앱의 기본 정보, 구성 요소를 등록하는 파일
 - 앱 아이콘, 이름, 사용자가 보는 화면들, 처리할 요청들, 권한 등
 - 예시: 시스템 공유 옵션에 등록하기, 인터넷 권한 부여하기

Android Studio 프로젝트 구조

- res 폴더
 - 앱에서 사용하는 아이콘, 그림 등 리소스 파일
- AndroidManifest.xml
 - OS에 앱의 기본 정보, 구성 요소를 등록하는 파일
 - 앱 아이콘, 이름, 사용자가 보는 화면들, 처리할 요청들, 권한 등
 - 예시: 시스템 공유 옵션에 등록하기, 인터넷 권한 부여하기

안드로이드 앱의 구성 요소

- 안드로이드 프레임워크
- Android Studio 프로젝트 구조
- 안드로이드 앱의 구성 요소 4가지
- Activity Lifecycle
- Context

안드로이드 앱의 구성 요소

- Android OS가 생성하고 실행할 수 있는 기능 단위
- 무엇이 있을까요?

안드로이드 앱의 구성 요소

- Activity
 - '화면'
 - Service
 - '백그라운드 작업'
 - 음악 프로그램 - 노래 들려주기
 - 게임 - 대용량 파일 다운로드
 - Broadcast Receiver
 - 시스템, 앱 이벤트 수신
 - 배터리가 없으면 이렇게 행동해라
 - ContentProvider
 - 다른 앱에 정보 제공
 - 연락처 - 타 앱에서 연락처 연동 가능
- 세미나에서는 대부분의 시간 동안 Activity 1개 제작하는 법을 다룹니다

안드로이드 앱의 구성 요소

- 안드로이드 프레임워크
- Android Studio 프로젝트 구조
- 안드로이드 앱의 구성 요소 4가지
- Activity Lifecycle
- Context

Activity Lifecycle

- 우리가 하는 일
 - ComponentActivity에 Cmd + B
 - 윈도우는 Ctrl + B
 - Activity를 상속받기
 - 일부 함수를 override하기
 - 우리 앱 진입점이라고
AndroidManifest에 등록하기

```
1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Seminar_1Theme {
                Scaffold(modifier = Modifier.fillMaxSize() {
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(16dp)
                    )
                }
            }
        }
    }
}
```

Activity Lifecycle

- Activity가 생성되면 onCreate
 - 화면이 보이기 시작하면 onStart
 - 다른 화면에 의해 가려지면 onPause
 - 화면이 완전히 안 보이면 onStop
 - 제거되면 onDestroy
-
- 예시) 화면을 회전하면, Activity가 사라졌다가 다시 생긴다
 - Activity가 들고 있던 값들이 전부 사라진다

안드로이드 앱의 구성 요소

- 안드로이드 프레임워크
- Android Studio 프로젝트 구조
- 안드로이드 앱의 구성 요소 4가지
- Activity Lifecycle
- Context

Context

- 앱이 OS의 어떤 환경에서 실행되고 있는지를 담은 정보
 - OS가 앱에 맞춰 프로세스, 자원을 그때그때 할당하기 때문에 매번 바뀝니다
- Toast를 띄울 때, 다른 화면을 열 때, 문자열 리소스를 가져올 때 등
- Activity Context, Application Context

```
Toast.makeText(  
    this,  
    "저장되었습니다",  
    Toast.LENGTH_SHORT  
).show()
```

(액티비티 내부에서)

오늘 할 일

- 오리엔테이션
- AI 시대의 개발
 - 개발이 무엇인가요?
 - AI로 개발하기
- Git에 대해서
- Kotlin 코드 이해하기
- 안드로이드 앱의 구성 요소
- 과제 안내

과제 안내